Thursday 6:47 PM

https://github.com/SETAPESU26/21_checkers

I checked the repository. This is your **Scenario 21 – Checkers** lab. GitHub repository ↗

The important part is that **you are not supposed to rewrite the project from scratch**. The README explicitly says to inspect the existing code, fix the existing defect, implement Tasks 2–4, test the edge cases, and retain the complete LLM chat history. GitHub

## What you need to complete

| Task | What you need to implement |
|---|---|
| **1. Capture correctness** | When a piece jumps an opponent, remove the captured piece |
| **2. Win/no-move detection** | End game if a player has no pieces **or** no legal moves |
| **3. Complete rules** | Forced captures, multi-captures, kings moving both directions, correct promotion |
| **4. Move feedback** | Print one clear result for each accepted move |

The required tests include ordinary moves, captures, multiple captures, forced captures, promotion, king movement, no-move positions, invalid coordinates, and quitting. GitHub

## Your repository currently contains

Plain text

```
21_checkers/
├── README.md
├── requirements.txt
├── main.py
├── game.py
├── board.py
└── rules.py
```

The README identifies `main.py` as the entry point, `game.py` as the turn/interaction logic, `board.py` as board representation/movement, and `rules.py` as the legal-move/promotion logic. GitHub

**I can help you complete this entire lab step-by-step.** The best next step is for you to **upload the ZIP/project files here** (or upload `main.py`, `game.py`, `board.py`, and `rules.py`). Then I can inspect the actual code and tell you exactly what to change, rather than guessing at the implementation.

lets do it step by step

Absolutely. We'll do it **one task at a time**, and I'll explain what we're doing before giving you the code.

## Step 1 — Set up and inspect the project

First, download/clone the repository:

GitHub — 21_checkers ↗

If you're using VS Code:

1. Open the repository folder in VS Code.
2. Open the terminal.
3. Run:

```bash
python --version
```

Then:

```bash
pip install -r requirements.txt
```

And finally:

```bash
python main.py
```

## Don't modify anything yet

For **Step 1**, I want to see the original code exactly as provided.

Please upload these four files here:

```
Plain text
main.py
game.py
board.py
rules.py
```

(or simply upload the **ZIP** of the repository).

Once you upload it, we'll start with **Task 1: Capture correctness** and I'll walk you through:

**current code → what's wrong → why it's wrong → exact change → run test → expected output.**

Then we'll move to Task 2, 3, and 4 in order.

First, download/clone the repository: how do i do it

Sure. Since you're on **Windows + VS Code**, the easiest way is with Git.

## Option 1 — Clone using VS Code (recommended)

### 1. Install Git first

Open this official page:

Git for Windows ↗

Install it using the default options.

### 2. Open VS Code

Open VS Code and press:

```
Ctrl + Shift + P
```

Type:

```
Plain text
Git: Clone
```

Select **Git: Clone**.

### 3. Enter the repository URL

Paste:

next

Great. Let's continue.

## Step 2 — Install the project's requirements

In the VS Code terminal, make sure you're still here:

```
Plain text
PS C:\Users\prith\Documents\GitHub\21_checkers>
```

Run:

```
powershell
python -m pip install -r requirements.txt
```

Wait for it to finish.

You should ideally see something like:

```
Plain text
Successfully installed ...
```

## Step 3 — Run the original Checkers program

After the installation finishes, run:

```
powershell
python main.py
```

**Don't change any code yet.**

Tell me exactly what appears after `python main.py` — you can paste the terminal output or send a screenshot.

Then we'll inspect how the original program behaves **before fixing Task 1**.

```
[notice] A new release of pip is available: 26.0.1 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
PS C:\Users\prith\Documents\GitHub\21_checkers> python main.py
Checkers — move: sr sc er ec

  0 1 2 3 4 5 6 7
0 . B . B . B . B
1 B . B . B . B .
2 . B . B . B . B
3 . . . . . . . .
4 . . . . . . . .
5 R . R . R . R .
6 . R . R . R . R
7 R . R . R . R .
R>
```

Worked for 12s

Great, Prithviraj! 🎉 Your Checkers game is running successfully. The pip update notice is harmless; you don't need to update pip.

Now let's start **Task 1: Fix capture correctness**.

## Step 3 — Understand the bug

In VS Code, your game is currently running in the terminal.

### 1. Exit the game

Type:

```
q
```

Press **Enter**.

### 2. Open `board.py`

In the Explorer on the left, click `board.py`.

You should find this function:

Python

```
def move_piece(board, start, end):
    board[end[0]][end[1]] = board[start[0]][start[1]]
    board[start[0]][start[1]] = "."
```

### 3. Understand what's wrong

Suppose the board contains:

Plain text

```
R . B .
```

Red jumps over the blue piece. The expected result is:

Plain text

```
. . . R
```

But the existing function only moves the red piece. It **doesn't remove the captured blue piece**.

We need to fix that.

### 4. Replace the function

Replace the existing `move_piece()` function with this code:

Python

```
def move_piece(board, start, end):
    sr, sc = start
    er, ec = end

    piece = board[sr][sc]

    # Remove the captured piece when a jump occurs
    if abs(er - sr) == 2 and abs(ec - sc) == 2:
        mr = (sr + er) // 2
        mc = (sc + ec) // 2
        board[mr][mc] = "."

    # Move the piece to its destination
```

```
        board[er][ec] = piece
        board[sr][sc] = "."
```

**Save the file** using `Ctrl + S`.

### Why does this work?

When a piece jumps two rows and two columns, the captured piece is located halfway between the starting and ending positions.

For example, a move from `(5, 0)` to `(3, 2)` captures the piece at `(4, 1)`.

The code calculates that middle position and removes the captured piece.

---

### Your turn

Complete these three actions:

1. Exit the game by typing `q`.
2. Replace `move_piece()` in `board.py`.
3. Save the file.

**Tell me when you've finished.** Next, we'll test whether the capture works correctly before moving to Task 2.

done

Perfect. 👍

### Step 4 — Test Task 1

Let's make sure the capture fix actually works **before moving on**.

In the VS Code terminal, run:

```
powershell
python main.py
```

You'll get the board again.

We need to make a capture move. The starting board is:

Plain text

```
   0 1 2 3 4 5 6 7
0  . B . B . B . B
1  B . B . B . B .
2  . B . B . B . B
3  . . . . . . . .
4  . . . . . . . .
5  R . R . R . R .
6  . R . R . R . R
7  R . R . R . R .
R>
```

At this starting position, there isn't an immediate capture for Red, so first make a normal move.

Enter:

Plain text

```
5 0 4 1
```

Red moves from `(5,0)` → `(4,1)`.

Then Black should get its turn.

### Important

Don't worry if the exact prompt/output looks slightly different.

**Send me a screenshot of the board after this move.**

Then I'll give you the exact next move to create a capture and we'll verify that the captured piece disappears.

next

Worked for 6s

Good. Since you said **next**, let's continue without making you manually figure out the moves.

### Step 4 — Test the capture fix

Run:

powershell

```
def __init__(self):
    self.board = initial_board()
    self.player = "R"

def print_board(self):
    print("\n   " + " ".join(str(c) for c in range(SIZE)))
    for r, row in enumerate(self.board):
        print(f"{r}  " + " ".join(row))

def run(self):
    print("Checkers — move: sr sc er ec")
    while True:
        self.print_board()
        raw = input(f"{self.player}> ").strip().lower().split()
        if raw == ["q"]:
            return
        if len(raw) != 4:
            print("Enter four coordinates.")
            continue
        try:
            sr, sc, er, ec = map(int, raw)
        except ValueError:
            print("Coordinates must be numbers.")
            continue
        if not all(0 <= x < SIZE for x in (sr, sc, er, ec)):
            print("Outside board.")
            continue
        if self.board[sr][sc] not in (self.player, self.player + "K"):
            print("That is not your piece.")
            continue

        start, end = (sr, sc), (er, ec)
        if capture_move(self.board, self.player, start, end):
            move_piece(self.board, start, end)
        elif simple_move(self.board, self.player, start, end):
            move_piece(self.board, start, end)
        else:
```

```
            print("Invalid move.")
            continue

        promote(self.board)
        self.player = "B" if self.player == "R" else "R"
```

Perfect. Now we can work with the **actual code** instead of guessing.

Your `game.py` currently has **no win/no-move check**. After every move it simply does:

```python
promote(self.board)
self.player = "B" if self.player == "R" else "R"
```

So even if the opponent has no pieces or no legal moves, the game continues.

### Before we modify `game.py`

I need to see how your existing move-validation functions work, because Task 2 depends on them.

Please open **`rules.py`** and paste its complete contents here.

Then I'll give you the exact Task 2 changes — likely we'll add a function such as:

```python
has_legal_move(...)
```

and then integrate it into `game.py`.

**Don't change `game.py` yet.**

```
SIZE = 8


def simple_move(board, player, start, end):
    sr, sc = start
    er, ec = end
    direction = -1 if player == "R" else 1
    return (
```

```
        board[er][ec] == "." and
        abs(er - sr) == 1 and abs(ec - sc) == 1 and
        er - sr == direction
    )


def capture_move(board, player, start, end):
    sr, sc = start
    er, ec = end
    direction = -1 if player == "R" else 1
    mr, mc = (sr + er) // 2, (sc + ec) // 2
    return (
        board[er][ec] == "." and
        abs(er - sr) == 2 and abs(ec - sc) == 2 and
        er - sr == 2 * direction and
        board[mr][mc] not in (".", player)
    )


def promote(board):
    for c in range(SIZE):
        if board[0][c] == "R":
            board[0][c] = "RK"
        if board[SIZE - 1][c] == "B":
            board[SIZE - 1][c] = "BK"
```

Perfect. Now we have the actual `rules.py`, so we can implement **Task 2 correctly**.

There are two things we need to detect:

1. **No pieces left** → the player loses.
2. **Pieces exist, but no legal move** → the player loses.

For now, we'll work with the movement rules that already exist. **Kings and forced captures will be handled in Task 3.**

---

## Step 5 — Add no-move detection

Open `rules.py`.

At the **bottom** of the file, after `promote()`, add:

Python

```
def has_pieces(board, player):
    return any(
        cell == player or cell == player + "K"
        for row in board
        for cell in row
    )


def has_legal_move(board, player):
    for r in range(SIZE):
        for c in range(SIZE):
            if board[r][c] not in (player, player + "K"):
                continue

            # Check all possible destination squares
            for er in range(SIZE):
                for ec in range(SIZE):
                    if simple_move(board, player, (r, c), (er, ec)):
                        return True

                    if capture_move(board, player, (r, c), (er, ec)):
```

So your `rules.py` should now end like this:

Python

```
def promote(board):
    for c in range(SIZE):
        if board[0][c] == "R":
            board[0][c] = "RK"
        if board[SIZE - 1][c] == "B":
            board[SIZE - 1][c] = "BK"


def has_pieces(board, player):
    return any(
        cell == player or cell == player + "K"
        for row in board
        for cell in row
    )
```

```
def has_legal_move(board, player):
    for r in range(SIZE):
        for c in range(SIZE):
            if board[r][c] not in (player, player + "K"):
```

Save with:

```
Plain text
Ctrl + S
```

---

## Step 6 — Modify `game.py`

At the top you currently have:

```
Python
from rules import simple_move, capture_move, promote
```

Change it to:

```
Python
from rules import simple_move, capture_move, promote, has_pieces, has_legal_move
```

Then find the bottom of your `run()` method:

```
Python
promote(self.board)
self.player = "B" if self.player == "R" else "R"
```

Replace those two lines with:

```
Python
promote(self.board)

# Switch player
```

```
    self.player = "B" if self.player == "R" else "R"

    # Check whether the new player can continue
    if not has_pieces(self.board, self.player):
        winner = "B" if self.player == "R" else "R"
        self.print_board()
        print(f"{winner} wins! {self.player} has no pieces.")
        return

    if not has_legal_move(self.board, self.player):
        winner = "B" if self.player == "R" else "R"
        self.print_board()
        print(f"{winner} wins! {self.player} has no legal moves.")
        return
```

---

## What we just accomplished

The game now does:

```
Plain text
```

```
Red moves
   ↓
Promote if necessary
   ↓
Switch to Black
   ↓
Does Black have pieces?
   ↓
NO → Red wins
   ↓
YES
   ↓
Does Black have a legal move?
   ↓
NO → Red wins
   ↓
YES
   ↓
Continue game
```

## Don't worry about this yet

You may notice that `has_legal_move()` doesn't properly account for **king movement**.

That's intentional.

**Task 3 will fix:**

- Kings moving both directions
- Forced captures
- Multiple captures
- Correct promotion
- Complete checkers rules

So don't try to fix those now.

### Step 7 — Test

Run:

```powershell
python main.py
```

If it starts normally, that's a good sign.

**Don't try to manually create a no-move position yet.** Just tell me whether:

```
python main.py
```

starts successfully.

Then we'll proceed to **Task 3 — Complete Checkers rules**, which is the biggest part.

after replacing those 2 lines its throwing an self is not defined

Yes — that's my mistake. The code was probably pasted **outside the `run()` method**, so Python says `self` is not defined.

Let's fix it cleanly.

### Your `game.py` should look like this

Replace the **entire contents of `game.py`** with:

Python

```
from board import initial_board, move_piece, SIZE
from rules import simple_move, capture_move, promote, has_pieces, has_legal_move


class Checkers:
    def __init__(self):
        self.board = initial_board()
        self.player = "R"

    def print_board(self):
        print("\n   " + " ".join(str(c) for c in range(SIZE)))
        for r, row in enumerate(self.board):
            print(f"{r}  " + " ".join(row))

    def run(self):
        print("Checkers — move: sr sc er ec")

        while True:
            self.print_board()

            raw = input(f"{self.player}> ").strip().lower().split()
```

## Important

Notice the indentation:

Python

```
            # Switch player
            self.player = "B" if self.player == "R" else "R"

            if not has_pieces(...):
                ...
```

Everything involving `self` must be **inside `def run(self):`**.

Save:

Plain text

```
Ctrl + S
```

Then run:

```powershell
python main.py
```

### If it starts

You should get:

```
Checkers — move: sr sc er ec

   0 1 2 3 4 5 6 7
0  . B . B . B . B
...
R>
```

If you get **any error**, paste the complete error here. Don't change anything else.

nxt

Good. Let's move to **Task 3 — Complete Checkers Rules**.

This is the main part. We need to fix four things:

1. **Kings can move in both directions**
2. **Captures are compulsory**
3. **Multiple captures are possible**
4. **Promotion works correctly**

### First: don't change anything yet

We need to modify `rules.py`, but I want to do it carefully because your current code treats normal pieces and kings differently.

Open `rules.py`.

You currently have:

```python
def simple_move(board, player, start, end):
```

and

Python

```
def capture_move(board, player, start, end):
```

We'll replace these functions with versions that understand kings.

## Step 1 — Replace `simple_move()`

Replace only your current `simple_move()` with:

Python

```
def simple_move(board, player, start, end):
    sr, sc = start
    er, ec = end

    piece = board[sr][sc]

    # Destination must be empty
    if board[er][ec] != ".":
        return False

    # Must move exactly one diagonal square
    if abs(er - sr) != 1 or abs(ec - sc) != 1:
        return False

    # Kings can move in either direction
    if piece == player + "K":
        return True

    # Normal pieces can only move forward
    direction = -1 if player == "R" else 1
    return er - sr == direction
```

## Step 2 — Replace `capture_move()`

Replace your current function with:

Python

```
def capture_move(board, player, start, end):
    sr, sc = start
    er, ec = end

    piece = board[sr][sc]

    # Destination must be empty
```

```
    if board[er][ec] != ".":
        return False

    # Capture must be exactly two diagonal squares
    if abs(er - sr) != 2 or abs(ec - sc) != 2:
        return False

    # Middle square contains the captured piece
    mr = (sr + er) // 2
    mc = (sc + ec) // 2

    if board[mr][mc] in (".", player, player + "K"):
        return False
```

## Step 3 — Save

Press:

Plain text

```
Ctrl + S
```

**Don't test yet.**

Next we'll implement **forced captures**, because that's a separate rule and it requires changing `game.py`.

Just tell me **"done"** after you've replaced these two functions.

Yesterday 4:27 PM

done

Good. Now we'll implement the next part of **Task 3: forced captures**.

## Step 4 — Add a function to detect captures

Open `rules.py`.

At the bottom, **after your existing functions**, add:

Python

```
def has_capture(board, player):
    for r in range(SIZE):
        for c in range(SIZE):
```

```
            if board[r][c] not in (player, player + "K"):
                continue

            for er in range(SIZE):
                for ec in range(SIZE):
                    if capture_move(board, player, (r, c), (er, ec)):
                        return True

    return False
```

This asks:

"Does this player have **any capture available anywhere on the board**?"

---

## Step 5 — Make captures compulsory

Now open `game.py`.

Change this import:

```python
from rules import simple_move, capture_move, promote, has_pieces, has_legal_move
```

to:

```python
from rules import (
    simple_move,
    capture_move,
    promote,
    has_pieces,
    has_legal_move,
    has_capture
)
```

Now find this section:

```python
```

```
if capture_move(self.board, self.player, start, end):
    move_piece(self.board, start, end)

elif simple_move(self.board, self.player, start, end):
    move_piece(self.board, start, end)
```

Replace it with:

Python

```
if has_capture(self.board, self.player):
    if capture_move(self.board, self.player, start, end):
        move_piece(self.board, start, end)
    else:
        print("You must capture when a capture is available.")
        continue

elif simple_move(self.board, self.player, start, end):
    move_piece(self.board, start, end)

else:
    print("Invalid move.")
    continue
```

## What this does

Before every move:

Plain text

```
Does the player have a capture?
        |
       YES
        ↓
Only a capture is allowed
        |
       NO
        ↓
Normal move is allowed
```

So a player **cannot ignore an available capture**.

Save both files:

```
Plain text
Ctrl + S
```

### Don't test multiple captures yet

We still need to implement **multi-capture**, where:

```
Plain text
R jumps B
    ↓
another capture is available
    ↓
R must jump again
    ↓
turn remains with R
```

That's the next step.

Tell me "**done**" once you've added `has_capture()` and changed the `game.py` section.

self not defoned

Yes — same issue as before: the new block was probably pasted at the wrong indentation level. Let's avoid piecemeal editing.

**Replace the entire `game.py` with this exact version.** This keeps all the previous changes and adds forced-capture correctly.

```
Python
from board import initial_board, move_piece, SIZE
from rules import (
    simple_move,
    capture_move,
    promote,
    has_pieces,
    has_legal_move,
    has_capture
)


class Checkers:
    def __init__(self):
        self.board = initial_board()
```

```
        self.player = "R"

    def print_board(self):
        print("\n   " + " ".join(str(c) for c in range(SIZE)))
        for r, row in enumerate(self.board):
            print(f"{r}  " + " ".join(row))
```

### Then save

Press:

```
Plain text
Ctrl + S
```

Then run:

```
powershell
python main.py
```

### If you get an error

Don't try to fix it yourself. **Copy the complete red error message and send it to me.**

If it starts with:

```
Plain text
Checkers — move: sr sc er ec
```

then we're good and we'll test **forced capture** next.

```
  0 1 2 3 4 5 6 7
0 . B . B . B . B
1 B . B . B . B .
2 . B . R . B . B
3 . . . . . . . .
4 . . . . . . . .
5 . . R . R . R .
6 . R . R . R . R
```

```python
was_capture = capture_move(
    self.board,
    self.player,
    start,
    end
)
```

Then after:

```python
promote(self.board)
```

**before switching players**, add:

```python
if was_capture and has_piece_capture(self.board, self.player, end):
    print("Another capture is available. Continue with the same piece.")
    continue
```

So this section becomes:

```python
promote(self.board)

if was_capture and has_piece_capture(self.board, self.player, end):
    print("Another capture is available. Continue with the same piece.")
    continue

# Switch player
self.player = "B" if self.player == "R" else "R"
```

## One more change

Update your import at the top of `game.py` :

```python
from rules import (
    simple_move,
```

```
    capture_move,
    promote,
    has_pieces,
    has_legal_move,
    has_capture,
    has_piece_capture
)
```

Save both files.

---

### What we've implemented

Now:

```
Red captures Black
       ↓
Check same Red piece
       ↓
Another capture available?
       ↓
YES → Red moves again
       ↓
NO → switch to Black
```

That's the **multi-capture rule**.

Don't test it manually yet.

Tell me **"done"** after making these changes, and we'll do the final part of Task 3: **correct king behavior + promotion testing**.

Yesterday 11:35 PM



he sent final pr how do i approve

Yes — you're on the **Pull Request page**, and it is currently **awaiting approval**.

From your screenshot:

```
siddhardha-06 wants to merge 3 commits into main from docs/final
```

and on the right:

**No reviews — at least 1 approving review is required.**

### To approve it

You're currently near the top of the PR. **Scroll down** the page.

Look for the **Files changed** tab/section. Click:

**Files changed**

Then review the changes. After that, scroll **all the way to the bottom** of the Files changed page.

You should see a **Review changes** button.

Click:

**Review changes → Approve → Submit review**

If GitHub asks for a comment, you can write:

```
Plain text
Reviewed and approved.
```

Then click **Submit review**.

### Important

Don't click **Merge pull request** yourself unless you're also responsible for merging it.

Your job here is just:

**Review → Approve → Submit review**

After you click **Submit review**, send me a screenshot and I'll tell you the next exact button to click.

lets continue

Yep. Let's continue with the **Checkers project**.

Before we add multi-capture, we need to make sure the helper functions are actually present.

Your `rules.py` should currently contain these functions:

```
Plain text
simple_move()
capture_move()
promote()
has_pieces()
has_legal_move()
has_capture()
has_piece_capture()
```

Open `rules.py` and check the bottom.

If **`has_piece_capture()` is already there**, we're ready for the next step.

If it's **not there**, add:

```
Python
def has_piece_capture(board, player, position):
    r, c = position

    for er in range(SIZE):
        for ec in range(SIZE):
            if capture_move(board, player, position, (er, ec)):
                return True

    return False
```

## Then check `game.py`

At the top, you should have:

```
Python
from rules import (
    simple_move,
    capture_move,
    promote,
    has_pieces,
    has_legal_move,
    has_capture,
```

```
    has_piece_capture
)
```

And after a successful move, you should have:

Python

```
promote(self.board)

if was_capture and has_piece_capture(self.board, self.player, end):
    print("Another capture is available. Continue with the same piece.")
    continue

self.player = "B" if self.player == "R" else "R"
```

**Don't change anything else.**

If those pieces are already present, tell me **"present"** and we'll move directly to the final Task 3 testing.

promote(self.board)

if was_capture and has_piece_capture(self.board, self.player, end):
print("Another capture is available. Continue with the same piece.")
continue

self.player = "B" if self.player == "R" else "R"
where shouldi paste this

Paste it **inside** `run()`, immediately after the move has been made and **before** `self.player` switches.

To avoid indentation mistakes, let's do it using your current `game.py`.

Find this part:

Python

```
        promote(self.board)

        # Switch player
        self.player = "B" if self.player == "R" else "R"
```

Replace **those three lines** with:

Python

```
promote(self.board)

# Check for another capture by the same piece
if was_capture and has_piece_capture(self.board, self.player, end):
    print("Another capture is available. Continue with the same piece.")
    continue

# Switch player
self.player = "B" if self.player == "R" else "R"
```

## BUT one important thing

You also need to create `was_capture` earlier.

Find:

Python

```
start = (sr, sc)
end = (er, ec)
```

Immediately **below it**, paste:

Python

```
was_capture = capture_move(
    self.board,
    self.player,
    start,
    end
)
```

So that section should look like:

Python

```
start = (sr, sc)
end = (er, ec)

was_capture = capture_move(
```

```
            self.board,
            self.player,
            start,
            end
        )

        # Captures are compulsory when available
        if has_capture(self.board, self.player):
```

### Finally, make sure the import contains `has_piece_capture`

At the very top:

```python
from rules import (
    simple_move,
    capture_move,
    promote,
    has_pieces,
    has_legal_move,
    has_capture,
    has_piece_capture
)
```

Save with **Ctrl + S**.

Then run:

```powershell
python main.py
```

If you get **any red error**, paste it here rather than changing anything else.

```
from board import initial_board, move_piece, SIZE
from rules import (
    simple_move,
    capture_move,
    promote,
    has_pieces,
    has_legal_move,
    has_capture,
```

If it starts with:

```
Plain text
Checkers — move: sr sc er ec
```

we've fixed the syntax/ `self` problem.

**Don't make any moves yet.** Just tell me whether `python main.py` starts successfully.

```
R> python main.py
Enter four coordinates.

  0 1 2 3 4 5 6 7
0 . B . B . B . B
1 B . B . B . B .
2 . . . B . B . B
3 . . B . . . . .
4 . R . . . . . .
5 . . R . R . R .
6 . R . R . R . R
7 R . R . R . R .
R> python main.py
Enter four coordinates.

  0 1 2 3 4 5 6 7
0 . B . B . B . B
1 B . B . B . B .
2 . . . B . B . B
3 . . B . . . . .
4 . R . . . . . .
5 . . R . R . R .
6 . R . R . R . R
7 R . R . R . R .
R>
```

You're fine — **`game.py` is running**. The reason you're seeing:

```
Plain text
R> python main.py
```

is that you typed `python main.py` **inside the Checkers game**.

The game is waiting for a move at:

```
Plain text
R>
```

So `python main.py` is being interpreted as your move, which produces:

```
Plain text
Enter four coordinates.
```

## Do this now

At the current:

```
Plain text
R>
```

type:

```
Plain text
q
```

This exits the Checkers game and returns you to PowerShell:

```
Plain text
PS C:\Users\prith\Documents\GitHub\21_checkers>
```

Then run:

```
powershell
```

```
python main.py
```

You should see:

```
Checkers — move: sr sc er ec

  0 1 2 3 4 5 6 7
...
R>
```

**Important:** Once you see `R>`, don't type `python main.py`. That's the game prompt. Use `q` to exit it.

After it starts correctly, we'll test the **multi-capture** functionality.

yes,nxt

Perfect. Now let's **finish Task 3** rather than keep doing random board moves.

## Step 8 — Test the multi-capture logic

Start the game:

```
python main.py
```

At the initial `R>` position, make:

```
5 0 4 1
```

Then Black:

```
2 1 3 2
```

Now Red should be able to capture:

```
Plain text
4 1 2 3
```

After that capture, **look at the prompt**.

If another capture is available for the same Red piece, it should say:

```
Plain text
Another capture is available. Continue with the same piece.
```

and the prompt should remain:

```
Plain text
R>
```

rather than changing to `B>`.

### One important correction

Our current `move_piece()` implementation removes the jumped piece, but **multi-capture can expose another capture only if the board position actually creates one**. So if you don't get the message after the above sequence, that doesn't necessarily mean the code is broken.

What matters for this step is that the program **doesn't crash** and the capture is performed correctly.

---

## After this: Task 4

Once we've confirmed Task 3 doesn't crash, we'll move to the final requirement:

### Task 4 — Move feedback

We'll make the program print a clear result such as:

```
Plain text
Move accepted: (5,0) -> (4,1)
```

and for captures:

Plain text

```
Capture: (4,1) -> (2,3)
```

Then we'll run through the required edge cases and clean up the project.

**Do the three moves above and tell me what appears after `4 1 2 3`.**

```
  0 1 2 3 4 5 6 7
0 . B . B . B . B
1 B . B . B . B .
2 . B . B . B . B
3 . . . . . . . .
4 . . . . . . . .
5 R . R . R . R .
6 . R . R . R . R
7 R . R . R . R .
R> 5 0 4 1
```

```
  0 1 2 3 4 5 6 7
0 . B . B . B . B
1 B . B . B . B .
2 . B . B . B . B
3 . . . . . . . .
4 . R . . . . . .
5 . . R . R . R .
6 . R . R . R . R
7 R . R . R . R .
B> 2 1 3 2
```

```
  0 1 2 3 4 5 6 7
0 . B . B . B . B
1 B . B . B . B .
2 . . . B . B . B
3 . . B . . . . .
4 . R . . . . . .
5 . . R . R . R .
```

```
6 . R . R . R . R
7 R . R . R . R .
R> 4 1 2 3
Invalid move.
```

The `Invalid move` is actually revealing a **real bug in our implementation**. The Black piece is at `(3,2)`, so `(4,1) → (2,3)` should be a valid capture.

The problem is in `board.py`: our `move_piece()` is probably not the only place involved—the current `capture_move()` logic needs to be checked against the actual board code.

**Let's stop changing things blindly.**

## Do this

Exit the game:

```
Plain text
q
```

Then open **`board.py`** and paste the **entire contents** here.

I want to inspect the actual `move_piece()` implementation before we make another change. Then I'll give you the exact fix and we can test the capture properly.

```
SIZE = 8


def initial_board():
    board = [["."] * SIZE for _ in range(SIZE)]
    for r in range(3):
        for c in range(SIZE):
            if (r + c) % 2 == 1:
                board[r][c] = "B"
    for r in range(5, 8):
        for c in range(SIZE):
            if (r + c) % 2 == 1:
                board[r][c] = "R"
    return board
```

```python
SIZE = 8


def simple_move(board, player, start, end):
    sr, sc = start
    er, ec = end

    piece = board[sr][sc]

    # Destination must be empty
    if board[er][ec] != ".":
        return False

    # Normal move is one diagonal square
    if abs(er - sr) != 1 or abs(ec - sc) != 1:
        return False

    # King can move in either direction
    if piece == player + "K":
        return True
```

Save:

```
Ctrl + S
```

## Then test again

Run:

```powershell
python main.py
```

Do exactly:

```
5 0 4 1
```

then:

```
Plain text
2 1 3 2
```

then:

```
Plain text
4 1 2 3
```

This time the capture **should be accepted**.

The resulting board should have:

```
Plain text
2 . . . R . B . B
3 . . . . . . . .
```

Most importantly, the Black piece at `(3,2)` must disappear.

If it works, **don't make any further changes**. Tell me what appears after the third move, and we'll proceed to the next rule.

```
B> 2 1 3 2

  0 1 2 3 4 5 6 7
0 . B . B . B . B
1 B . B . B . B .
2 . . . B . B . B
3 . . B . . . . .
4 . R . . . . . .
5 . . R . R . R .
6 . R . R . R . R
7 R . R . R . R .
R> 4 1 2 3
Invalid move.
```

Okay — **stop here. Don't change any more code yet.**

The board position proves the move *should* be valid:

```
Plain text
```

```
Red:   (4,1)
Black: (3,2)
Empty: (2,3)

        Black
         (3,2)
        ↙
Red (4,1)  →  (2,3)
```

So if you're still getting `Invalid move`, the problem is that **the `rules.py` currently being executed is not behaving the way we expect**. Let's diagnose the actual saved file instead of guessing.

## 1. Exit the game

At:

```
Plain text
```

```
R>
```

type:

```
Plain text
```

```
q
```

## 2. In PowerShell, run this exact command

```
powershell
```

```
python -c "from rules import capture_move; b=[['.']*8 for _ in range(8)]; b[4][1]='R'; b[3][2]='B'; print(capt
```

## Expected result

It **must** print:

Plain text

```
True
```

### If it prints `False`

Then your `rules.py` is still not the version we expect, and we'll fix that directly.

### If it prints `True`

Then the problem is in `game.py`, specifically the move-processing logic, and we'll fix that.

**Run that one command and send me exactly what it prints.** Don't change anything else yet.

true

Perfect. That tells us something important:

Plain text

```
capture_move(...) → True
```

So **`rules.py` is correct**.

The problem is in `game.py`.

### Let's find exactly where

Exit the game if you're still in it:

Plain text

```
q
```

Then run this command:

powershell

```
python -c "from game import Checkers; g=Checkers(); g.board[4][1]='R'; g.board[3][2]='B'; print(g.board[2][3])
```

Actually, let's not complicate it. We already know the rule function works.

### The real issue

Look at your `game.py` logic:

```python
if has_capture(self.board, self.player):
    if was_capture:
        move_piece(self.board, start, end)
```

`has_capture()` is checking whether **any** capture exists for Red.

But your starting position may already have another capture elsewhere, and `was_capture` is being calculated before the move. That shouldn't make the shown move invalid, though.

So I want to see the **exact current `game.py`**, because we've modified it several times.

**Paste your entire current `game.py` here.**

Don't change anything first. I'll identify the exact line causing `Invalid move` and give you a corrected version.

```
if has_capture(self.board, self.player):
        if was_capture:
            move_piece(self.board, start, end)
        else:
            print("You must capture when a capture is available.")
            continue
```

Yes — **that block itself is correct**.

The issue is likely that `was_capture` is being calculated incorrectly or the surrounding code isn't exactly what we think.

Let's simplify this so there is less chance of another bug.

## Replace this:

```python
if has_capture(self.board, self.player):
    if was_capture:
        move_piece(self.board, start, end)
    else:
```

```
        print("You must capture when a capture is available.")
        continue
```

## With this:

```python
if has_capture(self.board, self.player):
    if capture_move(self.board, self.player, start, end):
        move_piece(self.board, start, end)
    else:
        print("You must capture when a capture is available.")
        continue

elif simple_move(self.board, self.player, start, end):
    move_piece(self.board, start, end)

else:
    print("Invalid move.")
    continue
```

## And remove the `was_capture` variable entirely.

So remove this too:

```python
was_capture = capture_move(
    self.board,
    self.player,
    start,
    end
)
```

## Your move section should now be exactly:

```python
start = (sr, sc)
end = (er, ec)

# Captures are compulsory when available
if has_capture(self.board, self.player):
    if capture_move(self.board, self.player, start, end):
```

```
            move_piece(self.board, start, end)
        else:
            print("You must capture when a capture is available.")
            continue

    # If no capture is available, allow a normal move
    elif simple_move(self.board, self.player, start, end):
        move_piece(self.board, start, end)

    else:
        print("Invalid move.")
        continue
```

Save with **Ctrl+S**.

Then run:

```
powershell
```

```
python main.py
```

Test:

```
Plain text
```

```
5 0 4 1
2 1 3 2
4 1 2 3
```

This should now accept the capture.

### One more thing

If it **still** says `Invalid move`, don't change anything else. Send me the **entire current `game.py`**, because at that point I want to correct the whole file in one shot rather than keep patching individual blocks.

Today 10:04 AM

give complete code of main.py

Yes. For your project, `main.py` should be very simple because the actual game logic is in `game.py`.

Replace the **entire contents of `main.py`** with:

Python

```
from game import Checkers


def main():
    game = Checkers()
    game.run()


if __name__ == "__main__":
    main()
```

Save with **Ctrl + S**.

Then run:

powershell

```
python main.py
```

You should get:

Plain text

```
Checkers — move: sr sc er ec

   0 1 2 3 4 5 6 7
0  . B . B . B . B
1  B . B . B . B .
2  . B . B . B . B
3  . . . . . . . .
4  . . . . . . . .
5  R . R . R . R .
6  . R . R . R . R
7  R . R . R . R .
R>
```

**Important:** `main.py` does not need any of the capture/movement logic. Keep that in `game.py` and `rules.py`.

If `python main.py` gives an error after this, paste the **complete error message**, and we'll fix that next.

i mean game.py

Yes — you mean the **complete** `game.py`. My mistake.

Replace the **entire contents of** `game.py` with this version:

```python
from board import initial_board, move_piece, SIZE
from rules import (
    simple_move,
    capture_move,
    promote,
    has_pieces,
    has_legal_move,
    has_capture,
    has_piece_capture
)


class Checkers:
    def __init__(self):
        self.board = initial_board()
        self.player = "B"

    def print_board(self):
        print("\n   " + " ".join(str(c) for c in range(SIZE)))
        for r, row in enumerate(self.board):
            print(f"{r}  " + " ".join(row))
```

## Save it

Press:

```
Ctrl + S
```

Then exit any currently running game with:

```
q
```

and run:

```powershell
python main.py
```

### One important thing

This `game.py` assumes your `rules.py` contains these functions:

```
simple_move()
capture_move()
promote()
has_pieces()
has_legal_move()
has_capture()
has_piece_capture()
```

which is the version we've been building.

**Don't make any moves yet.** First run `python main.py`. If it starts normally, tell me "**started**", and we'll test the capture again.

started

Great. ✅ Now let's test the exact bug we were chasing.

### Test 1 — Normal move

At:

```
R>
```

enter:

```
5 0 4 1
```

2. Make sure the repository is:

   **`21_checkers`**
3. You should see your changed files:
   - `board.py`
   - `game.py`
   - `rules.py`
4. On the bottom-left, enter a commit message such as:

   **`Complete checkers lab implementation`**
5. Click **Commit to main**.
6. Then click **Push origin** at the top.

### Important

Don't modify the code anymore right now. Your syntax check passed:

```
Plain text
python -m py_compile main.py game.py board.py rules.py
```

So the next step is simply to **commit and push your working changes through GitHub Desktop**.

If you want, you can also send me a screenshot of your GitHub Desktop window and I'll tell you **exactly what to click**.



Yes — I can see the issue clearly. 👀

GitHub Desktop shows **7 changed files**, but only these **3 files should be committed**:

- ✅ `board.py`
- ✅ `game.py`
- ✅ `rules.py`

The other 4 are Python-generated cache files:

- ❌ `__pycache__/board.cpython-313.pyc`
- ❌ `__pycache__/game.cpython-313.pyc`
- ❌ `__pycache__/main.cpython-313.pyc`
- ❌ `__pycache__/rules.cpython-313.pyc`

`Prithviraj-techie/21_checkers-1`

After the push completes, the up-arrow should disappear.

## After pushing

We'll still need to do the remaining assignment deliverables:

- 🎥 Before video
- 🎥 After video
- 📁 `Lab-4` folder
- 💻 Updated code
- 📄 Exported ChatGPT conversation as PDF/DOC

But first **click** **`Push origin`** and tell me when it finishes.

pushed

Perfect. ✅ Your four commits are now pushed to your **personal fork**.

Now the remaining Lab-4 deliverables are:

1. 🎥 **Before video** — 10 seconds of the original broken game
2. 🎥 **After video** — 10 seconds of the completed game
3. 💻 **Updated code** — already pushed to `Prithviraj-techie/21_checkers-1`
4. 📄 **Chat history exported as DOC/PDF**
5. 📁 Everything needs to be under a **`Lab-4`** **folder** in your personal lab repository.

## One important thing

Your assignment says:

> **Do not raise PR to the main repo ( `SETAPESU26` )**

So **don't create a PR**. You're done with the code push.

## Before/after videos

If you already recorded the **before** video, great.

For the **after** video, run:

</> powershell

```
cmd

python main.py
```

3. Click **inside the Command Prompt window** so it becomes the active window.
4. Make sure the Checkers game is visible.
5. Press:
   **Win + Alt + R**
6. Record the game for ~10 seconds.
7. Press **Win + Alt + R** again to stop.

### If Game Bar still records the wrong thing

Use **Windows Snipping Tool** instead:

1. Press **Win + Shift + R**
2. Drag around the **Command Prompt window**
3. Click **Start**
4. Record the Checkers terminal.
5. Click **Stop** when finished.

For your assignment, the important thing is that the video clearly shows the **Checkers game running**, not ChatGPT.

**Tip:** Put the Command Prompt and game on screen, then don't click Chrome while recording.

nzxt

Next, let's finish the **video part correctly**.

### Record the AFTER video again

1. Open **Command Prompt**.
2. Go to your repo:

```
cmd

cd C:\Users\prith\Documents\GitHub\21_checkers
```

3. Run:

```
cmd
```

into:

```
Plain text

21_checkers
└── Lab-4
    └── Chat_History.pdf
```

## Important

Because your assignment says:

> **Chat history exported as a doc/pdf**

make sure the PDF contains the **actual conversation**, including the prompts you gave ChatGPT and the responses—not just a summary.

After you've saved the PDF, tell me **"PDF done"** and we'll finish the `Lab-4` folder and push the final deliverables.